

Drum Rig

A procedural 16-pad drum machine and step sequencer

Luca Franzin

Interactive Graphics — Prof. Marco Schaerf
DIAG, Sapienza University of Rome

Contents

1	Overview	4
1.1	What it does	4
2	Environment	4
2.1	Three.js rather than raw WebGL	4
2.2	Libraries and the import map	4
3	Third-party inventory	5
4	Architecture	5
4.1	The module graph	5
4.2	Keeping modules decoupled	5
4.3	The event bus	6
4.4	The event vocabulary	6
4.5	The render loop	7
4.6	Per-module responsibilities	9
5	The hierarchical model	9
5.1	The slab: joint tree	9
5.2	The constraint	10
5.3	What the constraint drives	10
5.3.1	Rotation.	11
5.3.2	Hinge translation.	11
5.3.3	Pad retraction.	11
5.4	The fold sequence	11
5.5	Interruptible motion	11
5.6	Rob8: the second hierarchical model	12
5.7	Two animations sharing one chain	12
5.7.1	The stroke (hitting a drum):	12
5.7.2	The groove (bouncing to the beat):	13
5.8	Phase: smooth motion synced to the beat	13
5.9	Pad press: why exponential decay is better than a tween	13

6	The audio engine	13
6.1	Synthesis rather than samples	13
6.1.1	Envelopes are exponential and never hit zero.	14
6.1.2	Velocity controls brightness, not just volume.	14
6.1.3	Repeated hits are slightly randomized.	14
6.1.4	Nodes are built fresh per hit and discarded.	14
6.2	The master bus	14
6.2.1	Drive	14
6.2.2	Space	15
6.2.3	Tone	15
6.3	Why four knob mappings are exponential	15
6.4	The lookahead scheduler	15
6.4.1	Absolute time vs. accumulated time.	16
6.4.2	Tempo changes.	16
6.4.3	Different layers.	16
6.5	Swing	16
6.6	Deferred visual events	16
6.7	Choking	17
6.8	Layers, mute and solo	17
6.9	Recording	17
6.9.1	Quantization.	18
6.9.2	Trigger vs Audition.	18
6.10	Pattern shape and undo	18
7	Lighting	18
7.1	The static rig	19
7.1.1	Exactly one shadow caster.	19
7.2	Reading the spectrum	19
7.2.1	fftSize is 2048, not the default 1024.	19
7.2.2	Bin indices are calculated dynamically.	19
7.3	Three bands, and the asymmetry hidden inside them	20
7.3.1	A noise gate at 0.06.	20
7.4	Asymmetric envelope following	20
7.5	Placement derived from a bearing	20
7.6	Colour: one clock, fixed offsets	21
7.7	Volumetric shafts: why the second implementation	21
7.8	The pass, step by step	22
7.8.1	Occlusion needs a depth prepass.	22
7.8.2	The interval is clipped to the sphere the light reaches.	22
7.8.3	The starting offset is dithered per pixel.	22
7.8.4	Dark beams exit early.	22
7.8.5	The angular falloff is squared.	22
7.8.6	Loop bounds are compile-time constants.	22
7.9	What is approximated	22
8	Materials and textures	23
8.1	The seven kinds	23
8.2	The pipeline	23
8.2.0.1	The blur wraps rather than clamps.	23
8.3	Five material families, and what distinguishes them	24
8.4	The toon ramp.	24

8.5	Canvas textures	24
8.6	The mask	25
8.7	Image-based lighting	25
8.8	Emissive	25
8.9	Material types in the scene	26
9	User manual	26
9.1	Getting started	26
9.2	The instrument	26
	9.2.1 Hover readouts.	27
	9.2.2 Dragging a knob disables the camera.	27
9.3	Keyboard	27
9.4	The step grid	27
9.5	The control panel	28
9.6	Camera	28

1 Overview

Drum Rig is a browser-based drum machine and step sequencer, designed to look like a physical instrument in a 3D room, with the presence of a robot mascot named Rob8. When you turn the machine on, the main panel unfolds to reveal a 4×4 grid of drum pads and six knobs. When you click a pad or press a key, it plays a drum sound, lights up, pulses the stage lights, and triggers an animation of Rob8 hitting a drum.

Everything in the scene is generated entirely by code. I didn't import any 3D models, image textures, audio samples, or animation files. Every animation is driven purely by math, using basic easing functions, trigonometry, or exponential decay.

1.1 What it does

- **Sixteen synthesized sounds** across four kits (Studio, Reggaetón, Techno, and Drill). The kits just change the audio parameters, so if you record a beat, you can swap kits without losing the meaning of the pattern.
- **A four-layer step sequencer** You can register sounds using the sequencer to have more control featuring: mute/solo buttons for each layer and an undo function.
- **Six built-in patterns** that is programmed in standard drum notation.
- **A folding animation system** for the main instrument, using joint constraints to drive the unfolding animation smoothly.
- **Audio-reactive lighting**: An `AnalyserNode` reads the master audio output and uses it to drive the intensity and color of the stage lights, complete with volumetric light shafts.
- **A secondary 3D model** (Rob8), whose arm animations are triggered by the same events that light up the drum pads.
- **An adaptive quality controller** that monitors the frame rate and automatically adjust rendering settings based on the current device.

2 Environment

2.1 Three.js rather than raw WebGL

I chose to use Three.js instead of raw WebGL. The main reason is that using raw WebGL would mean writing basic engine features (like scene graphs, matrix math, and shadow maps) entirely from scratch.

Three.js scene graph handles the parent-child relationships required for hierarchical modeling, and its robust material system made it much easier to write code for procedural textures.

2.2 Libraries and the import map

All three of the dependencies are saved locally in the `/lib` folder. The import map in `index.html` tells the browser exactly where to find those local files:

```
"imports": {
  "three":          "./lib/three/build/three.module.js",
  "three/addons/":  "./lib/three/addons/",
  "@tweenjs/tween.js": "./lib/tween/tween.esm.js",
  "lil-gui":         "./lib/lil-gui/lil-gui.esm.js"
}
```

3 Third-party inventory

Component	Version	Licence	Used for
three.js	r185.1	MIT	Rendering, scene graph, materials, raycasting
lil-gui	0.21.0	MIT	The control panel
tween.js	25.0.0	MIT	Eased interpolation for the unfolding and camera moves

Table 1: All three are stored locally in `/lib`.

The only Three.js add-on used in the project is `OrbitControls`, which is used strictly for free camera navigation.

4 Architecture

The application's structure relies entirely on two things: the import graph and a central event bus. There is no dependency injection container or module registry. This section explains how both systems work.

4.1 The module graph

The project consists of 21 JavaScript modules in the `/src` folder, plus `index.html`.

The code is structured as a directed acyclic graph (DAG) that is divided into five layers. A module is only allowed to import from the layers below it; it never imports from its own layer or any layer above it.

Layer	Module	Imports from <code>/src</code>
L5	<code>main.js</code>	seventeen modules — every one below except <code>geometry</code> , <code>textures</code> , <code>presets</code>
L4	<code>hierarchy.js</code> <code>intro.js</code>	<code>events</code> , <code>rig</code> , <code>tweens</code> <code>environment</code> , <code>events</code> , <code>geometry</code> , <code>palette</code> , <code>textures</code> , <code>tweens</code>
L3	<code>rig.js</code> <code>mascot.js</code> <code>lighting.js</code> <code>ui.js</code>	<code>events</code> , <code>geometry</code> , <code>pads</code> , <code>palette</code> , <code>textures</code> <code>events</code> , <code>geometry</code> , <code>pads</code> , <code>palette</code> , <code>textures</code> <code>environment</code> , <code>events</code> , <code>geometry</code> , <code>palette</code> <code>camera</code> , <code>events</code> , <code>pads</code> , <code>presets</code>
L2	<code>environment.js</code> <code>audio.js</code> <code>sequencer.js</code> <code>interaction.js</code> <code>camera.js</code>	<code>palette</code> , <code>textures</code> <code>events</code> , <code>pads</code> <code>events</code> , <code>pads</code> <code>events</code> , <code>pads</code> <code>events</code> , <code>tweens</code>
L1	<code>textures.js</code>	<code>palette</code>
L0	<code>events.js</code> , <code>tweens.js</code> , <code>geometry.js</code> , <code>palette.js</code> , <code>pads.js</code> , <code>presets.js</code> , <code>quality.js</code> , <code>volumetrics.js</code>	<i>none</i>

Table 2: The complete internal dependency list.

4.2 Keeping modules decoupled

While it would be nice to say `main.js` handles every single import in the project, that just isn't practical. Instead, I followed a simpler, more useful rule:

Peer modules on the same layer never import each other. Imports only point downward into shared helpers (like values or geometry constructors), never sideways into another system's logic.

For example, `audio.js` doesn't import `rig.js`, and `rig.js` doesn't import `audio.js`. Neither of them talks to `lighting.js` or `mascot.js`. When they do import things, it's strictly utility files—like `rig.js` pulling colors from `palette.js`.

Everything works through the concept of subscribe/publish nodes. So you can easily add other elements in the project without having to rewire anything. Some example of the usefulness of this approach are:

- A drum pad lights up exactly the same way whether you click it, press a key, or the sequencer triggers it. They all just fire a single `pad:hit` event. This means that i can write a single visual response and connect it to multiple events
- Adding Rob8's arm swing was incredibly easy. Just subscribe to the existing hit events in one new file instead of having to modify a bunch of existing files.

4.3 The event bus

`events.js` is a tiny 51-line file with no external dependencies. It acts as a basic publish/subscribe system, using a `Map` to link event names to a `Set` of listening functions.

Here, I made two specific design choices. First, store listeners in a `Set` rather than an array, which prevents duplicate events and makes adding/removing listeners fast. Second, the `emit` function wraps every listener in its own `try/catch` block:

```
for (const fn of set) {  
  try { fn(payload); }  
  catch (err) { console.error('[bus] listener for "${event}" threw:', err); }  
}
```

This is crucial because if one listener has a bug and crashes, I don't want it to kill the entire render loop and leave the user staring at a black screen. This isolation ensures that a broken feature only breaks itself.

4.4 The event vocabulary

Eight modules send out events, and six modules listen for them. Table 3 shows the complete list. This table really serves as the core interface of the entire app.

Event	Emitted by	Subscribed by
<i>Intent — raw input, before it means anything</i>		
<code>pad:trigger</code>	interaction	main
<code>button:press</code>	interaction	main
<code>knob:change</code>	interaction	main, ui
<code>transport:toggle</code>	interaction	main
<code>case:toggle</code>	interaction	main
<code>camera:next</code>	interaction	main
<code>hover</code>	interaction	main
<i>Transport and voices</i>		
<code>pad:hit</code>	audio	rig, mascot, sequencer
<code>transport:step</code>	audio	rig, mascot, sequencer
<code>transport:start</code>	audio	main, rig, mascot, sequencer, ui
<code>transport:stop</code>	audio	main, rig, mascot, sequencer, ui
<code>transport:bpm</code>	audio	rig, mascot, ui
<code>transport:swing</code>	audio	—
<i>Recording and pattern state</i>		
<code>record:armed</code>	audio	main, rig, sequencer
<code>record:start</code>	audio	sequencer
<code>record:stop</code>	audio	sequencer
<code>record:hit</code>	audio	sequencer
<code>record:countin</code>	audio	sequencer
<code>layers:changed</code>	audio	sequencer
<code>history:changed</code>	audio	sequencer
<code>pattern:length</code>	audio	sequencer
<i>Scene state</i>		
<code>rig:fold</code>	hierarchy	main, ui
<code>kit:changed</code>	main	rig, ui
<code>started</code>	main	main
<code>intro:start</code>	intro	main
<code>intro:done</code>	intro	main
<code>intro:activate</code>	intro	—
<code>camera:moving</code>	camera	—
<code>camera:arrived</code>	camera	—
<code>lighting:ready</code>	lighting	—
<code>audio:bands</code>	lighting	—

Table 3: The full event vocabulary. Six events currently have no subscriber and are used only for debug purpose

There are two main takeaways from this list.

First, every event sent by `interaction.js` is handled exclusively by `main.js`. This is intentional: `interaction.js` only reports raw data (like "knob one was moved to 0.42"), but `main.js` actually decides what that means.

Second, `audio.js` sends out the most events, and various other systems listen to it. But `audio.js` never imports them, and they never import it. This means the audio scheduler just emits a drum hit and moves on, it doesn't need to know that a light is about to flash or a robot is about to swing its arm.

4.5 The render loop

There is only one main loop in `main.js`, driven by `renderer.setAnimationLoop`. Modules do not schedule their own frame callbacks.

```

function tick(timeMs) {
  const dt = Math.min(clock.getDelta(), 0.1);
  const t = timeMs / 1000;

  tweens.update(timeMs); // advance the state scalars
  hierarchy.update();    // two scalars -> every joint transform
  audio.update();        // release scheduled events whose time has come
  rig.update(dt);        // pad press and rim glow decay
  lighting.update(dt);   // spectrum -> band energies -> emitter intensities
  mascot.update(dt, t);  // idle, blink, arm swing
  intro.update(dt);      // impact dust
  cameraRig.update(dt);

  volumetrics.setBeams(lighting.beams);
  volumetrics.renderDepth(scene, camera);
  renderer.render(scene, camera);
  volumetrics.render(camera, t);

  quality.update(dt);
  updateStatus(dt);
}

```

The order of these functions matters. Everything that updates the scene state runs before we actually draw it, meaning the frame always shows the most up-to-date information. `hierarchy.update()` has to run right after `tweens.update()` so the animations are applied immediately. Similarly, `volumetrics.setBeams()` needs to run after `lighting.update()` so the light shafts use the most recent audio intensity data.

The three rendering calls are also ordered carefully. First render the depth map so the volumetric light shafts know where they are blocked. Then render the main lit scene, and finally, overlay the light shafts on top. The `quality.update(dt)` function runs last so it can accurately measure how long the entire frame took to process.

Two other small details in this block:

I used Three.js's `setAnimationLoop` instead of a standard `requestAnimationFrame`. It is WebXR-compatible, handles rescheduling automatically, and provides a timestamp that `tween.js` expects out of the box, keeping animations perfectly in sync.

I also clamped the time delta using `Math.min(clock.getDelta(), 0.1)`. If the user switches browser tabs, the animation loop pauses. When they come back, the delta might jump to 30 seconds. Without a clamp, all the exponential decays in the project would process that massive jump instantly, causing the lights to shut off and the camera to teleport wildly.

4.6 Per-module responsibilities

Module	Responsibility
<code>main.js</code>	Renderer, scene, static light rig, the single loop, all wiring
<code>lighting.js</code>	Analyser, band split, envelope following, the reactive fixture rig and speaker stacks
<code>rig.js</code>	Slab geometry, materials, the joint skeleton with pivot placement
<code>volumetrics.js</code>	Raymarched light-shaft post-process pass
<code>textures.js</code>	Every texture map, generated procedurally at load
<code>intro.js</code>	The power-on sequence
<code>pads.js</code>	Sixteen synthesised drum voices across four kits
<code>audio.js</code>	Context, transport, lookahead scheduler, layers, recording
<code>mascot.js</code>	Rob8: the second hierarchical model, beat-driven
<code>sequencer.js</code>	The step-grid DOM panel
<code>interaction.js</code>	Raycasting, keyboard, drag tracking — input to intent
<code>camera.js</code>	Spherical shot presets, transitions, follow-cam
<code>presets.js</code>	Authored patterns in drum notation, and the compiler
<code>quality.js</code>	Adaptive resolution controller
<code>geometry.js</code>	Rounded-box and rounded-cylinder primitives
<code>ui.js</code>	The lil-gui panel, as a view over bus state
<code>hierarchy.js</code>	The fold constraint solver and its sequence
<code>environment.js</code>	Floor, fog, image-based lighting, contact shadows
<code>palette.js</code>	The colour scheme and the pad luminance solver
<code>events.js</code>	The publish/subscribe bus
<code>tweens.js</code>	One shared tween group for the whole project

Table 4: The twenty-one modules

`tweens.js` is just nine lines that export a single custom **Group**. Instead of relying on `tween.js`’s default global group, managing it explicitly allows us to pause, clear, or step every animation in the project at exactly the same time.

5 The hierarchical model

This section covers the first core project requirement: hierarchical modeling. I built two hierarchical models for this project. The first is the tri-fold drum machine, which uses a constraint system to fold its wings. The second is Rob8, the robot mascot, who has an articulated arm chain driven by the audio tempo.

They demonstrate two different approaches. The folding case shows how a single variable can drive a mechanical linkage, while Rob8 shows how a single motion can be distributed down a complex joint chain. Both are built from basic 3D primitives, and neither relies on imported animation files.

5.1 The slab: joint tree

The main instrument is a folding case. The center section holds the sixteen drum pads and two side wings hinge inward over the top, meeting in the middle to protect the controls.

Two details in this tree are especially helpful.

First, `slabRoot` acts as a dedicated container for the whole instrument. If one want to mount the drum machine on a stand later, it need to just move `slabRoot` instead of repositioning dozens of individual pieces.

Second, the knobs are grouped underneath the wing hinges. This is exactly why a scene graph is so useful: when a wing folds 180 degrees and lifts up, its knobs automatically follow perfectly.

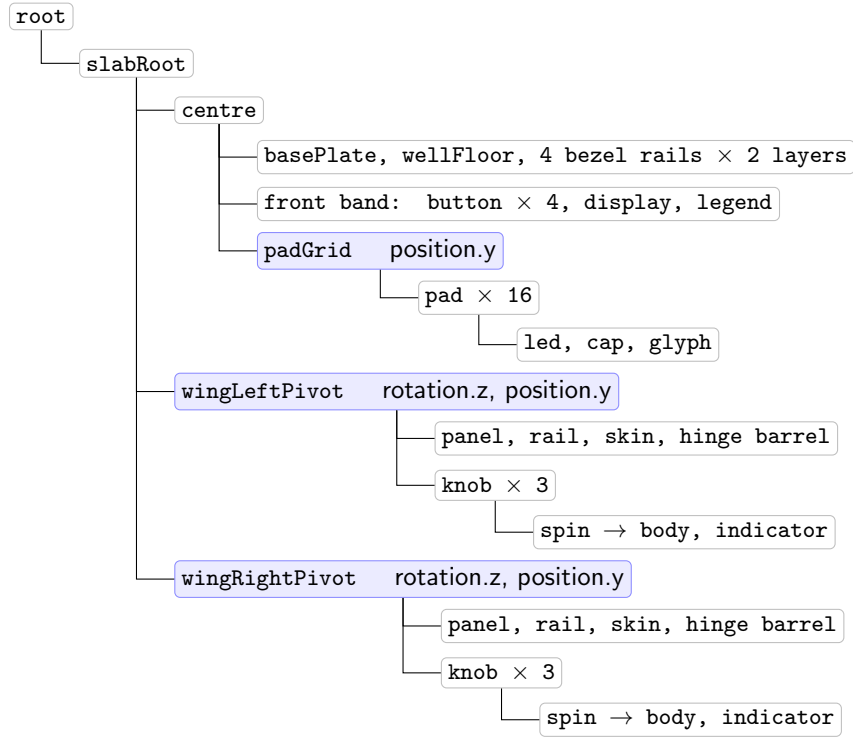


Figure 1: The drum machine’s scene graph. Shaded nodes represent joints — the only three objects `hierarchy.js` is allowed to touch. Everything else is static structure.

5.2 The constraint

The entire folding mechanism is driven by just two numbers: φ_L and φ_R , which represent the left and right wing angles. Everything else is calculated using a single mathematical function:

$$s(\varphi) = \sin^2\left(\frac{\varphi}{2}\right) \quad (1)$$

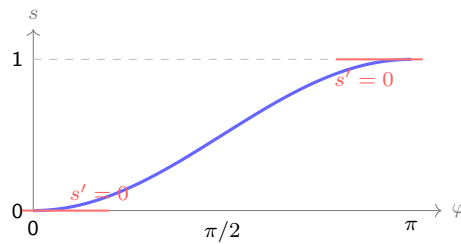


Figure 2: The shaping function. It always moves forward, and naturally smooths out at both ends.

I chose this specific function for a practical reasons:

its derivative hits zero at both ends. This means the animation naturally eases in and out on its own. The wings slow down before they finish opening or closing, which stops the animation from looking like it’s slamming into a wall.

5.3 What the constraint drives

As mentioned, `applyPose` is the only function that actually moves the model:

```
function applyPose(joints, state) {
  const { wingLeftPivot, wingRightPivot, padGrid } = joints;

  wingLeftPivot.rotation.z = -state.phiL;
  wingRightPivot.rotation.z = +state.phiR;

  wingLeftPivot.position.y = DIMS.slabsT + DIMS.hingeClear * hingeLift(state.phiL);
  wingRightPivot.position.y = DIMS.slabsT + DIMS.hingeClear * hingeLift(state.phiR);

  const lead = Math.max(state.phiL, state.phiR);
  padGrid.position.y = WELL_FLOOR_Y - DIMS.padTravel * 0.5 * hingeLift(lead);
}
```

This takes two angles and applies them to five different properties.

5.3.1 Rotation.

The left and right wings spin in opposite directions, handled simply by using a negative sign for the left.

5.3.2 Hinge translation.

Notice that it update `position.y` alongside the rotation. If the wing only rotate around it's axis then the thickness of the pads and the knobs will clip. Having the wing move slightly upwards will prevent that

5.3.3 Pad retraction.

As the case closes, the entire grid of drum pads sinks down into its tray. This is what allows the case to shut completely flat.

The code `'const lead = Math.max(state.phiL, state.phiR)'` is small but crucial.

Because the animated wings starts with a slight delay (one starts closing 260 ms before the other), the drum pads have to get out of the way of **both**. By checking the `'Math.max'` of the two angles, the pad grid starts sinking the millisecond the first wing begins to move, and it doesn't pop back up until both wings are fully open.

5.4 The fold sequence

I didn't just animate both wings opening at the exact same time, because it looks artificial. I added slight delays and different animation curves to give it a mechanical, physical feel.

Sequence	Joint	Delay	Duration	Easing
open	φ_R	0 ms	820 ms	Cubic.Out
	φ_L	260 ms	820 ms	Cubic.Out
shut	φ_L	0 ms	700 ms	Cubic.In
	φ_R	260 ms	700 ms	Cubic.In

Table 5: The folding sequences. Notice that the easing types and the order reverse depending on the direction.

5.5 Interruptible motion

If you click the fold button halfway through an animation, the wings cleanly reverse direction without snapping to a new position. This works naturally because of how I initialize the tweens:

```
stopActive(); // cancel tweens in flight
const from = { v: state[step.joint] }; // start from where it IS
```

Every animation starts exactly from the joint's current physical position, not from a hard-coded 0 or 100.

Also force the state to update once right before the very first frame renders:

```
update(); // at the end of initHierarchy, before anything renders
```

Because the 3D models are built in their "open" positions by default, this single line ensures the machine snaps to its closed state before the user sees anything. Otherwise, they would see an open drum machine flash on screen for a split second.

5.6 Rob8: the second hierarchical model

The robot mascot, Rob8, isn't just a static decoration. He is a completely separate 3D model with an arm chain. His animations are driven by the exact same `pad:hit` events that light up the drums.

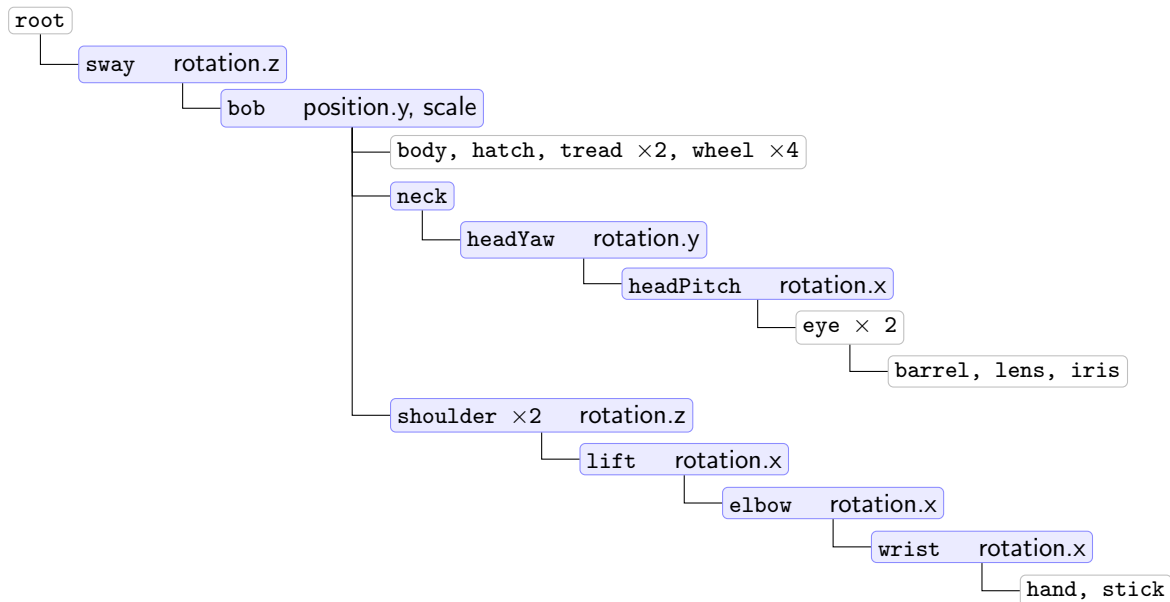


Figure 3: Rob8's bone chain.

5.7 Two animations sharing one chain

Rob8 should feel like a drummer instead of just twitching randomly given that his arms combine two separate movements.

5.7.1 The stroke (hitting a drum):

When a drum triggers, a single `strike` variable is distributed across three joints at once:

```
arm.lift.rotation.x = REST.lift + groove + arm.strike * 0.62;
arm.elbow.rotation.x = REST.elbow - groove * 0.45 - arm.strike * 0.38;
arm.wrist.rotation.x = REST.wrist - groove * 0.30 + arm.strike * 0.70;
```

Just like a real drum stroke, his upper arm drops, his elbow extends, and his wrist snaps down. The wrist gets the highest multiplier because the wrist is where a drummer generates the most speed.

5.7.2 The groove (bouncing to the beat):

Between hits, Rob8 counts the beat with his body. It use a simple sine wave to make his arms rise and fall to the tempo:

```
const phase = beat + (arm.side > 0 ? Math.PI : 0);  
const groove = Math.sin(phase) * grooveAmount * 0.20;
```

By offsetting one side by π (half a cycle), his left and right arms move exactly out of phase. Two arms moving together looks like a robot clapping; moving out of phase looks like a drummer grooving. The tempo data is pulled cleanly from the event bus via `transport:bpm`.

5.8 Phase: smooth motion synced to the beat

The animated Rob8's needs to correct his timing on the downbeat to never drift:

```
if (playing) beatPhase += dt * (bpm / 60);           // every frame  
  
bus.on('transport:step', ({ step }) => {  
  if (step % 4 === 0) beatPhase = Math.round(beatPhase); // downbeats only  
});
```

If I forced his arms to snap to a new position on exactly every 16th note, he would stutter mechanically. Instead, the animation runs continuously, and the event listener just nudges him back on-beat at the start of every measure.

5.9 Pad press: why exponential decay is better than a tween

The drum pad animation is small but highlights a practical optimization.

When you hit a pad, it dips 6 mm and lights up. If a user rapid-fires a drum pad at a fast tempo, it would have to constantly cancel and recreate tweens, which wastes performance. Instead, one can use simple exponential decay math:

```
pad.press *= Math.exp(-dt * PAD_EMISSIVE.capDecay);  
pad.group.position.y = pad.restY - pad.press * DIMS.padTravel;
```

If a pad gets hit again before it finishes rising, the variable just resets to 1 and decays naturally. No animations need to be canceled or re-allocated.

6 The audio engine

It features sixteen synthesized drum voices, a four-layer step sequencer, quantized recording with a count-in, and a master effects bus. More importantly, the audio engine drives almost every visual event in the scene—the drum pads light up, Rob8 swings his arms, and the stage lights pulse entirely based on audio timing events.

I split the logic into two files. `pads.js` handles the actual sound synthesis (the voices), while `audio.js` handles the timing and scheduling. This separation of concerns means that we can tweak how a snare drum sounds without ever touching the sequencer code.

6.1 Synthesis rather than samples

Every single drum sound is generated in real time using oscillators and filtered noise. I don't load any external '.wav' or '.mp3' files.

The main benefit of synthesizing sounds is that every voice is driven by mathematical parameters. When a user switches between the four drum kits, it don't load new files; just

swap out the numbers feeding the synthesizers. Because the underlying pad IDs never change, a user can record a beat and cycle through different kits without losing their pattern.

6.1.1 Envelopes are exponential and never hit zero.

```
param.setValueAtTime(0.0001, time);
param.exponentialRampToValueAtTime(top, time + attack);
param.exponentialRampToValueAtTime(0.0001, time + attack + decay);
```

Human hearing is logarithmic, so a linear fade-out sounds unnatural. I used exponential curves instead. However, because the Web Audio API throws a fatal error if you try to ramp an exponential curve to exactly zero, I faded down to a practically silent value of 10^{-4} .

6.1.2 Velocity controls brightness, not just volume.

```
function brightness(velocity, depth = 0.68) {
  return 1 - depth + depth * Math.pow(clamp01(velocity), 0.7);
}
```

When a real drummer hits a drum harder, it doesn't just get louder; the extra force excites higher frequencies, making the drum sound brighter. By mapping velocity to the filter cutoff as well, softer notes sound like light physical taps rather than just a volume slider being turned down. The added a 0.7 exponent to the math is needed to make the response feel natural.

6.1.3 Repeated hits are slightly randomized.

If you play the exact same waveform repeatedly at a fast tempo, it sounds like a machine gun rather than separate drum hits. To fix this, the noise-based sounds pull from a shared audio buffer at random offsets, and the oscillator-based sounds slightly jitter their pitch by about 2%. This subtle variation ensures a fast string of 16th-note hi-hats actually sounds like sixteen slightly distinct strikes.

The difference is subtle but over repeated hit one can hear the slight difference between the hits.

6.1.4 Nodes are built fresh per hit and discarded.

Web Audio `OscillatorNodes` are designed to be one-shot; you literally cannot restart them after calling `stop()`. So, allocating a small audio graph for every single drum hit and letting the browser garbage-collect it afterward is actually the intended way to use the API. This also allows long cymbal crashes to overlap naturally.

6.2 The master bus

Every drum hit—whether scheduled by the sequencer or played live by the user—is routed into a single master effects chain before it goes to the speakers.

6.2.1 Drive

is a distortion effect using a fixed tanh curve. Instead of changing the curve itself, turning the dial pushes the audio signal harder into it (pre-gain), while automatically lowering the output volume (makeup gain) to compensate. This makes the drums sound thicker and crunchier without just blowing out the user's speakers. I run this node at `oversample: '4x'` to prevent

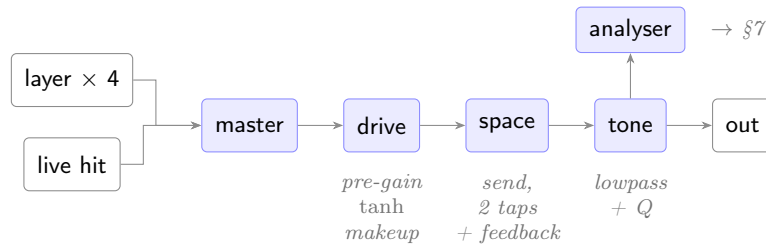


Figure 4: The master audio bus. The frequency analyser is tapped at the very end of the chain, ensuring the stage lights react to exactly what the user hears.

high-frequency artifacts (aliasing) from bouncing back down the spectrum and causing weird metallic ringing sounds.

6.2.2 Space

adds room reverberation built as a delay *send*. It uses two delay taps (131 ms and 187 ms) with a feedback loop and a damping filter.

6.2.3 Tone

is a standard lowpass filter placed at the very end of the chain, meaning it filters the reverb and distortion along with the raw drum sounds.

6.3 Why four knob mappings are exponential

$$f(t) = f_{\min} \left(\frac{f_{\max}}{f_{\min}} \right)^t, \quad 140 \text{ Hz} \leq f \leq 18 \text{ kHz} \quad (2)$$

Because humans perceive pitch logarithmically (jumping from 140 Hz to 280 Hz feels the same as jumping from 9 kHz to 18 kHz), mapping an audio frequency directly to a linear UI knob would do almost nothing for the first 75% of the rotation, and then wildly shift the sound at the very end.

By applying an exponential math curve, turning the knob by a physical millimeter always produces the same perceived musical change.

Whenever these knobs are moved, I apply the change using Web Audio's `setTargetAtTime` instead of a direct assignment. Assigning a value directly creates an instant jump in the audio signal which causes artifacts.

6.4 The lookahead scheduler

Getting a drum machine to stay on beat in the browser is not an easy task.

JavaScript's standard `setInterval` timer is wildly inaccurate and gets throttled by the browser if the user switches tabs. However, the Web Audio clock (`ctx.currentTime`) is perfectly sample-accurate.

To solve this, I used a lookahead scheduler. It uses a "planner" on the JavaScript thread to schedule notes on the Web Audio timeline ahead of time:

```

const LOOKAHEAD_MS = 25; // how often the planner wakes
const SCHEDULE_AHEAD = 0.1; // how far ahead it commits, in seconds

function planner() {
  const horizon = ctx.currentTime + SCHEDULE_AHEAD;
  const spb = secondsPerStep();

```

```

while (startTime + absStep * spb < horizon) {
  const time = startTime + absStep * spb + swingOffset(absStep, spb);
  scheduleStep(absStep, time);
  absStep++;
}
}

```

The planner doesn't play audio; it just guarantees that notes up to 100 ms in the future are locked into the audio thread.

6.4.1 Absolute time vs. accumulated time.

I calculate every step's time relative to the very beginning of the music (`startTime + n · spb`), rather than just adding the step duration to the previous note. If I added step-by-step, floating-point errors would slowly accumulate, and the beat would drift out of sync over a long session.

6.4.2 Tempo changes.

If a user turns the BPM knob, recalculating from the absolute 'startTime' would normally shift every previously scheduled note, breaking the song. To prevent this, whenever the tempo changes, it secretly rebase the 'startTime' backward so that the current step stays exactly where it is:

```

const nextTime = startTime + absStep * secondsPerStep();
bpm = clamped;
startTime = nextTime - absStep * secondsPerStep();

```

6.4.3 Different layers.

The `scheduleStep` function tracks each of the four sequencer layers independently based on their own loop length, meaning a 15-step pattern and a 16-step pattern can run side by side without requiring custom math.

6.5 Swing

The Swing knob is the most impactful musical control on the machine. Rather than applying an audio effect, it modifies the scheduler itself:

$$t(n) = \text{startTime} + n \cdot \text{spb} + \begin{cases} 0.62 \sigma \text{ spb} & n \text{ odd} \\ 0 & n \text{ even} \end{cases} \quad (3)$$

Perfectly straight 16th notes sound robotic. Human drummers naturally delay off-beats to create "groove." By dynamically pushing the start times of the odd-numbered steps later, it recreate that human feel. it is capped at a factor of 0.62; anything higher than that pushes the note too close to the next beat, making it sound broken rather than musical.

6.6 Deferred visual events

Because the audio scheduler commits notes up to 100 ms in the future, if the `pad:hit` event fired at schedule time, the drum pad would flash visually before the actual sound is even played.

To keep visuals and audio perfectly synchronized, I need to intercept scheduled visual events, queue them up with their audio timestamp, and release them during the main render loop exactly when the audio clock catches up:

```
while (pending.length && pending[0].time <= now) {
  const event = pending.shift();
  if (event.source === 'step') bus.emit('transport:step', event);
  else bus.emit('pad:hit', event);
}
```

Because of this queue, the main application (the pad flashes, the robot animation, the playhead) just listens for events normally, completely unaware of the lookahead logic.

6.7 Choking

In the real world, consecutive hits on the same drum will stop the previous sound. I implemented this by assigning voices to a `chokeGroup`. If a new note triggers, the engine finds what is currently ringing in that group and stops it.

The tricky part is stopping it at the correct scheduled time:

```
if (previous.gain.cancelAndHoldAtTime) previous.gain.cancelAndHoldAtTime(time);
else { /* cancelScheduledValues + setValueAtTime fallback */ }
previous.gain.exponentialRampToValueAtTime(0.0001, time + 0.02);
```

I use `'cancelAndHoldAtTime'` to freeze the volume exactly at the scheduled choke time, and then fade it out over 20 ms to prevent the harsh audio "pop" you get if you instantly silence it.

6.8 Layers, mute and solo

The sequencer has four tracks (layers) each with it's own gain. This allows each layer to change it's own volume without having any effect on the others.

```
const anySolo = layers.some((l) => l.solo);
for (const layer of layers) {
  const audible = anySolo ? layer.solo : !layer.muted;
  layer.effectiveGain = audible ? layer.gain : 0;
}
```

This guarantees that a Solo button inherently overrides a Mute button without requiring complicated logic.

6.9 Recording

The recording system is a small state machine with three states: disarmed, counting in, and actively capturing.

```
if (!running) {
  start();
  captureFromStep = countIn ? STEPS_PER_BAR : 0; // one bar of clicks
  capturing = !countIn;
} else {
  captureFromStep = Math.floor(currentAbsoluteStep());
  capturing = true; // tempo already audible
}
```

If you hit record while the machine is stopped, it gives you a one-bar metronome count-in before it starts capturing inputs, this is used to help the user hear the tempo before start playing. If you hit record while a beat is already playing, it starts capturing instantly, since you already hear the tempo.

6.9.1 Quantization.

When a player hits a pad, they are rarely perfectly on beat. So it needs to quantize their hit to the nearest 16th note step:

```
export function quantizeToStep(when) {
  const raw = (when - startTime) / secondsPerStep();
  const rounded = Math.round(raw);
  return ((rounded % patternLength) + patternLength) % patternLength;
}
```

I specifically use `Math.round` rather than `Math.floor`. Flooring would always drag a slightly late hit backward onto the previous step, making the entire recorded performance sound stange.

6.9.2 Trigger vs Audition.

When you strike a drum pad, it is an actual performance that gets recorded. But if you click a cell on the visual sequencer grid, you are just editing the pattern, and the audio only plays so you can preview what you placed. It use a single `fire()` function for both, checking a boolean flag to see if the sound should actually be captured by the recording system.

6.10 Pattern shape and undo

Patterns are stored as simple JSON objects:

```
{ length: 16, steps: [ [ { padId, velocity }, ... ], ... ] }
```

To have a simpler way to write the presets I wrote a small compiler that lets write presets using traditional drum-machine text notation:

kick_deep:	'X..x...X..x...'	X	accent	1.00
snare:	'...X.....X...'	x	normal	0.75
hat_closed:	'x.x.x.x.x.x.x.x.'	o	ghost	0.45

The compiler converts this notation into the exact same JSON format produced by the live sequencer, meaning the engine treats factory presets and user-recorded patterns exactly the same.

Undo functionality is handled with a single deep-copy snapshot. Before any major destructive action (like clearing a track or loading a new preset), it just take a copy of the sequence. If the user hits undo, simply replace the current JSON with the copy.

7 Lighting

The project uses two distinct lighting systems. A simple static light rig handles basic visibility and casts the scene's single shadow, while a reactive rig pulses and changes color based on the audio spectrum to create volumetric light shafts. This section covers both systems, as well as the custom raymarching shader written for the volumetric beams.

A key difference between this system and the 3D models is how they animate. The drum pads and the robot react to exact *events* (like `pad:hit`). However, the lights continuously react to the audio *spectrum*. They don't know when a pad is struck; they just monitor how much energy is currently in the bass, mid, and high frequencies. As a result, the light shafts swell and fade naturally with the music.

Also I intentionally kept `lighting.js` completely separate from `audio.js`. Browsers block the Web Audio API until the user clicks the screen, but the lights need to render immediately

on the very first frame. So keeping them separate allows to render the lights first.

7.1 The static rig

I kept the static rig simple: just four basic lights set up before any reactive effects take over.

Light	Type	Intensity	Job
hemisphere	HemisphereLight	0.55	Sky-to-ground ambient gradient, using the palette's own two colours
key	DirectionalLight	0.85	The only shadow caster. Warm, high and to the right
fill	DirectionalLight	0.16	Cool, opposite the key. Keeps shadows from going to pitch black
rim	DirectionalLight	0.34	Behind and left. Separates silhouettes from the background

Table 6: The static rig. This is a standard three-point lighting setup plus a basic ambient hemisphere.

7.1.1 Exactly one shadow caster.

I intentionally limited the scene to a single shadow-casting light. Shadow maps are computationally expensive.

To ground the objects, I added simple contact shadows under the instrument and the speakers. They are just dark radial-falloff textures placed slightly above the floor with `depthWrite` turned off convincing the eye that the machine is resting heavily on the concrete instead of floating.

7.2 Reading the spectrum

I pull frequency data from a single `AnalyserNode` placed at the very end of the master audio bus. This ensures the lights react to the final processed sound (including reverb and distortion) rather than just the raw, dry drum hits.

7.2.1 fftSize is 2048, not the default 1024.

The default FFT size is too small for accurate bass detection. Bumping the size to 2048 gives twice as much resolution in the low end.

7.2.2 Bin indices are calculated dynamically.

```
const binFor = (hz) => Math.round((hz * FFT_SIZE) / ctx.sampleRate);
```

Calculating this dynamically ensures the bass band always looks at the exact right frequencies regardless of the user's hardware (some hardware have different frequencies).

7.3 Three bands, and the asymmetry hidden inside them

Band	Range	Gain	Bins	Voices
bass	35–170 Hz	1.0	≈ 6	Both kicks, <code>sub_drop</code> , low tom body
mid	170–2200 Hz	1.5	≈ 90	Snare body, toms, rim, cowbell, clap
high	2200–12000 Hz	2.1	≈ 400	Hats, ride, crash, click

Table 7: the frequency cutoffs based on where the specific drum kits actually sit.

I applied a `BAND_GAIN` multiplier because low frequencies inherently carry way more energy than high frequencies. Without this correction, the bass light would always be pegged at maximum brightness, while the high-frequency lights would barely flicker.

However, the most important part of Table 7 is the bin count. FFT bins are spaced evenly, but audio frequency bands are logarithmic. This means the high band averages data across hundreds of bins, while the bass band only looks at about six. Because it’s a massive average, the high band’s signal is naturally smoother and steadier than the jumpy bass band. To make the high-frequency lights feel responsive, I had to apply faster animation speeds to them to compensate for this mathematical smoothing.

7.3.1 A noise gate at 0.06.

Without this, the ambient noise floor of the audio engine keeps the lights slightly glowing at all times.

7.4 Asymmetric envelope following

$$L_{t+\Delta t} = L_t + (T - L_t) \left(1 - e^{-k\Delta t}\right), \quad k = \begin{cases} k_{\text{attack}} & T > L_t \\ k_{\text{release}} & \text{otherwise} \end{cases} \quad (4)$$

```
bass: { attack: 26, release: 4.5 }  
mid:  { attack: 30, release: 6.0 }  
high: { attack: 60, release: 13.0 } // reciprocal seconds
```

It treat the attack (lighting up) and release (fading out) completely differently. The attack needs to be fast so it feels perfectly locked to the beat. The release needs to be slower so the human eye has time to register how bright the light got.

As mentioned earlier, the high-frequency band uses the fastest release time because it needs to flash rapidly enough to catch 16th-note hi-hats at 160 BPM without blurring into a solid beam of light.

Like other animations, it use a framerate-independent formula $(1 - e^{-k\Delta t})$ rather than basic linear interpolation. This guarantees the lights pulse and fade at the exact same physical speed whether the user is playing on a 60Hz laptop or a 144Hz monitor.

7.5 Placement derived from a bearing

eEvery light’s position, aim, and color are calculated dynamically from a single circular bearing around the center of the room:

```
apexAt(b) = [ sin(b) * RING.radius, RING.height, cos(b) * RING.radius ]  
aimFrom(b) = [ -sin(b) * RING.cross, RING.focusY, -cos(b) * RING.cross ]  
hueAt(b)   = (b / 2*PI) mod 1
```

This gave three structural benefits:

- All the lights sit perfectly on a circle, so every light shaft is identical in length and steepness.
- The rig is perfectly symmetrical from any camera angle.
- I can resize, raise, or tighten the entire rig by changing just one variable, and all the aiming logic follows perfectly.

I placed the lights high enough (3.60 units) so that their physical origins are completely out of frame on the main camera. Volumetric beams look much better when they originate off-camera; if you can see the glowing origin point, it shatters the illusion.

7.6 Colour: one clock, fixed offsets

The lights don't have individual, hardcoded colors. They all share the exact same saturation and lightness (0.88 and 0.55). It just shift their hue based on their position on the circle and a constantly running timer:

```
const programme = min(1, level.bass*0.6 + level.mid*0.5 + level.high*0.4);
hueClock = (hueClock + dt * (HUE_RATE + programme * HUE_PUSH)) % 1;

for (const f of fixtures)
    f.light.color.setHSL((hueClock + f.hue) % 1, BEAM_SATURATION, BEAM_LIGHTNESS);
```

Sharing the saturation and lightness makes the rig look cohesive, like a real concert lighting desk running a unified palette. Because each light has a fixed offset on the color wheel, they never accidentally turn the same color at the same time. The `HUE_RATE` smoothly cycles the colors around the room every 28 seconds.

Keeping the saturation high is crucial for the volumetric beams. Because they are rendered additively, a pure white beam just makes the background slightly brighter, which is hard for the eye to pick up. A highly saturated beam physically shifts the hue of whatever is behind it, making the light shaft pop clearly against the background.

7.7 Volumetric shafts: why the second implementation

I actually built the volumetric light shafts twice. The first version used basic transparent 3D cones. This looked terrible when two cones overlapped, because you could clearly see the hard polygonal edges of one cone clipping through the other. No alpha blending trick can fix the harsh line created by two overlapping triangle meshes.

Real haze doesn't have geometry edges; it's an integral of light scattering towards the eye:

$$L = \int \sigma \sum_i I_i(\mathbf{x}) dt \quad (5)$$

Because two crossing light beams just add their scattered light together naturally, the transition should be perfectly smooth. I fixed this by rewriting the effect as a custom raymarching shader. Now, when a blue beam and an amber beam cross, they smoothly create a brighter, warmer core instead of two overlapping polygons.

In the shader code, that entire concept boils down to this single line:

```
accum += sum * uIntensity[i] * stepLen * uDensity;    // sum computed per-sample
```

7.8 The pass, step by step

My custom raymarching shader calculates these shafts pixel by pixel:

7.8.1 Occlusion needs a depth prepass.

To prevent the light shafts from rendering straight through the solid drum machine, the shader needs to know where the physical objects are. Just do a cheap pre-pass of the scene using a basic depth material at half-resolution. This gives the raymarcher a map of where objects are so it knows exactly where to cut off the light beams. This is why the render loop generates the depth map first, then the lit scene, and then overlays the volumetrics.

7.8.2 The interval is clipped to the sphere the light reaches.

Since a cone mathematically goes on forever, the shader can waste a ton of performance. To fix this one can intersect the view ray with a bounding sphere around the light source. This ensures the shader only calculates math exactly where the light beam exists, massively improving performance.

7.8.3 The starting offset is dithered per pixel.

```
float dither = hash(vec3(gl_FragCoord.xy, uTime * 60.0));
float t = t0 + (float(s) + dither) * stepLen;
```

If every pixel started marching at the exact same distance, you would see concentric "slice" artifacts inside the light beam. By adding a tiny bit of random noise (dither) to the starting position, these slices turn into soft, high-frequency static. The human eye just perceives this static as dust or haze in the air. This trick lets us drastically lower the step count (saving GPU power) while keeping the beams looking great.

7.8.4 Dark beams exit early.

If a light is turned off or is extremely dim (`uIntensity[i] <= 0.002`), immediately skip it in the shader. This saves a huge amount of processing power during quiet parts of the song.

7.8.5 The angular falloff is squared.

I squared the math for the beam's soft edges so it perfectly matches Three.js's built-in `SpotLight` penumbra. This ensures the 3D beam hovering in the air visually matches the pool of light cast on the floor.

7.8.6 Loop bounds are compile-time constants.

Because WebGL 1.0 requires constant loop lengths, I hardcoded the maximum lights (6) and max raymarching steps (32), then use a runtime `break` to lower the quality dynamically if the framerate starts dropping on slower machines.

7.9 What is approximated

I made three intentional simplifications to keep the shader fast enough for real-time rendering:

Single scattering only. Assume light only bounces off the haze once. Simulating light bouncing around multiple times (multiple scattering) looks softer but requires incredibly complex, expensive techniques.

No shadowing within the volume. The light shafts don't have internal shadows. To make a physical object cast a dark shadow through the actual beam of light, one would have to sample shadow maps during every single step of the raymarch. The scene is optimized to use one shadow-casting light (the main key light), so one can accept perfectly clean, unshadowed beams here to save performance.

8 Materials and textures

I implemented seven distinct types of textures. However, there isn't a single image file in the entire repository. Every texture is generated procedurally at load time using noise algorithms and math filters.

8.1 The seven kinds

Kind	Slot	Space	What it carries
Base colour	map	sRGB	What colour the surface is
Normal	normalMap	linear	How light bends off it
Roughness	roughnessMap	linear	How sharp the reflection is
Emissive	emissive	sRGB	The pad caps and rims
Toon ramp	gradientMap	linear	How shading bands the light
Alpha mask	alphaMap	none	The contact-shadow falloff
Environment	environment	sRGB	Prefiltered studio reflections

Table 8: The seven texture types used, highlighting their color-space requirements.

This gives a lot of technical variety without just loading different JPG files.

8.2 The pipeline

All our noise-based textures are built using the exact same four-step pipeline, which helped keep the file size relative small.

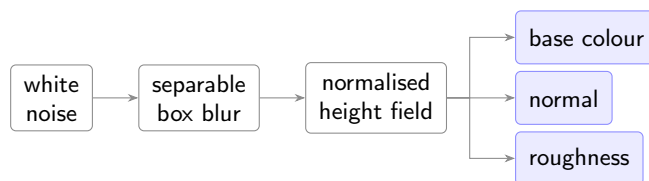


Figure 5: One height field generates three different maps. Also create different materials just by tweaking the blur radii and output ranges.

8.2.0.1 The blur wraps rather than clamps. When blurring the random noise, wrap the math around the edges of the texture instead of clamping it. This guarantees that all our textures tile seamlessly across large surfaces. I also used a separable box blur (doing two 1D passes instead of one heavy 2D pass) which makes the generation fast enough that it doesn't stutter the browser during load.

```

let nx = (left - right) * strength;
let ny = (up   - down ) * strength;
let nz = 1.0;
// normalise, then pack: v * 0.5 + 0.5

```

By comparing the height of neighboring pixels, I calculate the surface tilt. Normal maps only look like purple colors because the vector math ranges from $[-1, 1]$, but texture images can only store values between $[0, 1]$. So, I pack the math using $v/2 + 1/2$, and the shader naturally unpacks it later.

8.3 Five material families, and what distinguishes them

Family	Blur	Normal	Roughness	Used on
moulded plastic	3×3	0.5	0.62–0.78	Slab body, wings, buttons
rubber	1×1	1.1	0.70–0.90	Pad caps
brushed metal	14×1	0.9	0.30–0.55	Bezel frame, wing rails, hinges
concrete	two scales	0.55	0.34–0.68	The floor
painted wall	18×18	0.10	0.86–0.94	(unused — see below)

Table 9: By simply changing the blur radius and output ranges, I created completely different physical materials.

A few of these materials required specific tricks to look right:

Brushed metal: Blurred the noise heavily across the X-axis (radius 14) but barely at all on the Y-axis (radius 1). This creates long horizontal streaks that perfectly mimic the directional grooves of machined metal. Adding this so the instrument don't look like a solid block of plastic.

Rubber: Kept the blur extremely tight (radius 1). If you blur it any more, the surface starts looking like leather. A microscopic, harsh grain gives it that grippy silicone look.

Concrete: Concrete is tricky because it has two visual elements: tiny sandy aggregate up close, and large, sweeping stains from moisture. If you just blur noise once, it looks like plain sandpaper. So, I generated two separate noise fields (one fine, one coarse) and added them together.

8.4 The toon ramp.

```
toonRamp(levels = [0.45, 0.62, 0.80, 1.0]) // 4 bands, NearestFilter, no mipmaps
```

I intentionally gave Rob8 a cel-shaded look because he is a cartoon character, while the drum machine is a physical object. This visual separation prevents him from just looking like another piece of molded plastic standing in the room.

8.5 Canvas textures

I used the HTML5 2D Canvas API to generate four specific textures. While our noise maps define physical materials, these canvas textures display dynamic information.

Texture	Redrawn	Carries
glyphTexture	never	The key letter on each pad
textTexture	never	Silkscreen legends and the brand
knobCollarTexture	on knob change	A 270° value arc and the control's name
displayTexture	on transport events	Tempo, run state, current sixteenth, armed layer

The knob collar texture dynamically draws a colored arc around the knob based on its current value. I generate this arc using the exact same variable that physically rotates the 3D knob.

8.6 The mask

For the contact shadows underneath the drum machine, I generated a radial gradient mask. Interestingly, I wrote the gradient directly into the RGB color channels, and left the Alpha channel completely solid (255).

8.7 Image-based lighting

To make the environment lighting look realistic, I drew a panoramic sky gradient and some softbox lights to an equirectangular canvas, ran it through Three.js's `PMREMGGenerator`, and applied it to the scene.

An environment map gives the metal a full 360-degree room to reflect. As the camera orbits the drum machine, you can see the highlights roll across the rounded bezels.

The `PMREMGGenerator` is great because it pre-calculates different blur levels for the reflection. A highly polished metal part will sample the sharpest version of the room, while a rough concrete floor will sample a heavily blurred version automatically.

8.8 Emissive

The drum pad caps use the `emissive` property on a `MeshStandardMaterial`. Emissive materials ignore scene lighting and render totally flat. A translucent plastic button with an LED behind it doesn't have shadows; it glows uniformly in all directions.

If you just set 16 colors to the exact same HSL lightness value, they don't look equally bright to the human eye.

$$Y = 0.2126 R + 0.7152 G + 0.0722 B \tag{6}$$

Because humans are much more sensitive to green than blue, a green pad looks significantly brighter than a blue pad at the same math value.

To take account of this I wrote a small loop in `palette.js` that uses a binary search to find the exact HSL lightness needed for any given hue to hit a perceived target luminance of 0.60:

```
probe.setHSL(hue, PAD_SATURATION, mid);  
if (relativeLuminance(probe) < PAD_TARGET_LUMA) low = mid; else high = mid;
```

This perfectly balances all 16 colors.

8.9 Material types in the scene

Type	Where	Why that type
<code>MeshStandardMaterial</code>	Slab, pads, floor, cabinets	Physically based; the default for manufactured surfaces
<code>MeshToonMaterial</code>	Rob8 entirely	A drawn character, not a moulded part
<code>MeshBasicMaterial</code>	Legends, display, contact shadows	Printed ink and emissive panels are not lit surfaces — they have no form and no normal of their own

To prevent the text labels from glitching into the surface (z-fighting), I used `polygonOffset` rather than just lifting them higher on the Y-axis. If lifted them physically, they would look like they were floating above the metal when viewed from low camera angles.

9 User manual

This section explains how to use the drum machine and covers all the interactive features built. There are four different ways to control the project: clicking the 3D instrument directly, using the keyboard, using the sequencer grid at the bottom, or using the UI panel at the top right.

All of these inputs route through the exact same event system. This means if you change a setting via a keyboard shortcut, the 3D model and the UI panel will immediately update to match.

9.1 Getting started

When you first load the page, you'll see a screen with a **Power on** button. This isn't just for decoration—modern web browsers block the `AudioContext` from making sound until the user interacts with the page. So it requires one deliberate click so the audio engine is allowed to run.

Once clicked, the power-on animation plays. The instrument starts closed, with a small prompt at the bottom.

9.2 The instrument

You can click and interact with the 3D model directly.

Target	Gesture	Result
Pad	Left click	Plays the sound, lights the pad, and swings Rob8's arm. Records the hit if the layer is armed.
Knob	Left drag up/down	Adjusts the value. Dragging 220 px covers the full range.
Knob	Shift + drag	Fine adjustment (five times slower).
Button	Left click	Play/stop, record arm, or fold the machine.
Slab (shut)	Left click	Unfolds the case.
Empty space	Left drag	Orbits the camera around the room.
Anywhere	Scroll	Zooms the camera in and out.

9.2.1 Hover readouts.

If you hover over a pad or knob, a small tooltip appears showing its name and keyboard shortcut.

9.2.2 Dragging a knob disables the camera.

When you click a knob, instantly disable the camera orbit controls.

9.3 Keyboard

You can play the drum pads directly using the keyboard.

Key	Pad (Studio kit)	Key	Pad
1	Kick Deep	A	Rim
2	Kick Tight	S	Tom Low
3	Snare	D	Tom Mid
4	Clap	F	Tom High
Q	Hat Closed	Z	Click
W	Hat Open	X	Cowbell
E	Ride	C	Zap
R	Crash	V	Sub Drop

Table 10: The keyboard commands

Key	Action	Works when
Space	Start / stop the sequencer	Open
0	Unfold / fold the slab	Open or closed

9.4 The step grid

The panel at the bottom of the screen is the 2D step sequencer. It shows four layers, up to 64 steps, and a moving playhead.

Control	Gesture	Result
Cell	Left click	Writes the pad selected in <i>Draw</i> , and plays a quick preview sound.
Cell	Right click	Removes the selected pad from that step.
Arm (o)	Click	Arms that layer for recording.
M	Click	Mutes the layer.
S	Click	Solos the layer.
Clear (x)	Click	Clears the layer.
Play	Click	Starts or stops the music.
Record	Click	Arms the current row (gives a 1-bar count-in if stopped).
Undo	Click	Reverts to the last major snapshot.
Length	Select	Changes loop length to 1, 2, or 4 bars.
Draw	Select	Chooses which drum sound a left-click will place.
Collapse	Click	Hides the sequencer panel.

9.5 The control panel

The `lil-gui` panel in the top right is strictly a visual interface. It never stores data itself; it just reads from the event bus and updates its UI sliders to match.

Folder	Control	Notes
Transport	Play / Stop	Label updates based on the actual audio state.
	Tempo	40–200 BPM.
	Fold away / Unfold	Label follows the 3D model’s state.
Patterns	Six presets	Boom Bap, Four Four, Dembow, Four Floor Techno, Drill, Half Time.
	Clear all layers	
Camera	Six shots	Overview, Overview #2, Player, Rob8, Stage, Beams.
Master	Six sliders	Volume, Tone, Resonance, Drive, Space, Swing.
Kit	Four kits	Studio, Reggaetón, Techno, Drill.
	Loaded	Read-only label showing the active kit.

9.6 Camera

You can select six predefined camera angles from the UI. It store these shots using spherical coordinates relative to a target rather than hardcoded XYZ positions.

Shot	Framing
Overview	The default view. Shows the instrument, speakers, and Rob8.
Overview #2	A lower, front-facing angle to show off the wing folding mechanism.
Player	A top-down, close-up view of the drum pads.
Rob8	A close-up on the mascot to highlight his arm animations.
Stage	A wide shot showing the entire room setup.
Beams	A low angle looking up into the volumetric light shafts.